Defining Object-Relational Mapping

(https://github.com/learn-co-curriculum/python-p3-defining-object-relational-mapping) (https://github.com/learn-co-curriculum/python-p3-defining-object-relational-mapping/issues/new)

Learning Goals

- Create Python objects using SQL database records.
- Create SQL database records using Python objects.

Key Vocab

- **Object-Relational Mapping (ORM)**: a technique used to convert database records into objects in an object-oriented language.

Introduction

Object-Relational Mapping (ORM) is the technique of accessing a relational database using an object-oriented programming language. Object Relational Mapping is a way for our Python programs to manage database data by "mapping" database tables to classes and instances of classes to rows in those tables.

There is no special programming magic to an ORM — it is simply a manner in which we implement the code that connects our Python program to our database. For example, you can use code like this to connect your Python program to a given database:

```
db_connection = sqlite3.connect('db/my_database.db')
db_cursor = db_connection.cursor()
```

An ORM is really just a concept. It is a design pattern, a conventional way for us to organize our programs when we want those programs to connect to a database. The convention is this:

When "mapping" our program to a database, we equate **classes** with database **tables**, and **instances** of those classes with table **rows**.

You may also see this referred to as "wrapping" a database, because we are writing Python code that "wraps" or handles SQL.

Why Use ORM?

There are a number of reasons why we use the ORM pattern. Two good ones are:

- Cutting down on repetitive code.
- Implementing conventional patterns that are organized and sensical.

Cutting Down on Repetition

Let's take a look at some of the common code we might use to interact our Python program with our database.

Let's say we have a program that helps a veterinary office keep track of the pets it treats and those pets' owners. Such a program would have an `Owner` class and `Cat` class (among classes to represent other pets). Our program surely needs to connect to a database so that the veterinary office can persist information about its pets and owners.

Our program would create a connection to the database as we did above:

```
db_connection = sqlite3.connect('db/pets.db')
db_cursor = db_connection.cursor()
```

We would create an `owners` table and a `cats` table:

```
db_cursor.execute("CREATE TABLE IF NOT EXISTS cats(id INTEGER PRIMARY KEY, name TEXT, breed TEXT, age INTEGER)")

db_cursor.execute("CREATE TABLE IF NOT EXISTS owners(id INTEGER PRIMARY KEY, name TEXT)")
```

Then, we would need to regularly insert new cats and owners into these tables:

```
db_cursor.execute("INSERT INTO cats (name, breed, age) VALUES ('Maru', 'scottish fold', 3)")

db_cursor.execute("INSERT INTO cats (name, breed, age) VALUES ('Hana', 'tortoiseshell', 1)")
```

Notice that in the lines of code above, there is a lot of repetition. In fact, the only difference between the two lines in which we insert data into the database are the actual values.

The repetition would also occur for other SQL statements we might want to execute against our database. Any `SELECT` queries, for example, would repeat the call to the `db_cursor.execute` method and differ only in the specifics of what data we are selecting from which table.

As programmers, you might remember, we are lazy. We don't like to repeat ourselves if we can avoid it. Repetition qualifies as a "code smell". Instead of repeating the same, or similar, code any time we want to perform common actions against our database, we can write a series of methods to abstract that behavior.

For example, we can write a `save()` method on our `Cat` class that handles the common action of `INSERT`ing data into the database.

```
class Cat:

    all = []

    def __init__(self, name, breed, age):
        self.name = name
        self.breed = breed
        self.age = age
        self.add_cat_to_all(self)

    @classmethod
    def add_cat_to_all(cls, cat):
        cls.all.append(cat)

    def save(self, cursor):
        cursor.execute(
            'INSERT INTO cats (name, breed, age) VALUES (?, ?, ?)',
```

```
            (self.name, self.breed, self.age)
        )
```

Now let's create some new cats and save them to the database:

```
db_connection = sqlite3.connect('db/pets.db')
db_cursor = db_connection.cursor()

Cat("Maru", "scottish fold", 3)
Cat("Hana", "tortoiseshell", 1)

for cat in Cat.all:
    cat.save(db_cursor)
```

▶ *We only pass in a `cursor` as an argument for the `save()` method. Where does `save()` get the data to be inserted into the `cats` table?*

Here we establish the connection to our database, create two new cats and then iterate over our collection of cat instances stored in the `Cat.all` list. Inside this iteration, we use the `save()` method, giving it arguments of the data specific to each cat to `INSERT` those cat records into the cats table.

Now, thanks to our `save()` method, we have some re-usable code — code that we can easily use again and again to "save" or `INSERT` , cat records into the database.

This is just one example of the types of methods we will learn to build as we create our ORM. *Don't worry too much about the code shown above.* We'll learn more about how and why we define our ORM methods later on. This is just a preview of the kind of method we will write to tell our classes how to talk to our database.

Logical Design

Another important reason to implement the ORM pattern is that it just makes sense. Telling our Python program to communicate with our database is confusing enough without each individual developer having to make their own, individual decision about *how* our program should talk

to our database.

Instead, we follow the convention: classes are mapped to or equated with tables and instances of a class are equated to table rows.

If we have a `Cat` **class**, we have a `cats` **table**. `Cat` **instances** get stored as **rows** in the `cats` table.

Further, we don't have to make our own potentially confusing or non-sensical decision about what kinds of methods we will build to help our classes communicate with our database. Just like the `save()` method we previewed above, we will learn to build a series of common, conventional methods that our programs can rely on again and again to communicate with our database.

Conclusion

Object Relational Mapping is a common pattern in software design — every programming language has one or more popular libraries that help map between objects and relational databases to make developers' lives easier. Many of these ORMs, including SQLAlchemy, are built following similar conventions we've introduced here.

Resources

- **sqlite3 - DB-API 2.0 interface for SQLite databases - Python (https://docs.python.org/3/library/sqlite3.html)**
- **What is an ORM, how does it work, and how should I use one? - Stack Overflow (https://stackoverflow.com/questions/1279613/what-is-an-orm-how-does-it-work-and-how-should-i-use-one)**